

AGENTS AND PARALLEL WORK

Orchestrate teams of Claude Code sessions

Coordinate multiple Claude Code instances working together as a team, with shared tasks, inter-agent messaging, and centralized management.

⚠ Agent teams are experimental and disabled by default. Enable them by adding `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` to your settings.json or environment. Without that variable, no team is set up at session start, no team directories are written, and Claude does not spawn or propose teammates. Agent teams have known limitations around session resumption, task coordination, and shutdown behavior.

Agent teams let you coordinate multiple Claude Code instances working together. One session acts as the team lead, coordinating work, assigning tasks, and synthesizing results. Teammates work independently, each in its own context window, and communicate directly with each other.

Unlike subagents, which run within a single session and can only report back to the main agent, you can also interact with individual teammates directly without going through the lead.

ⓘ This page describes agent teams as of v2.1.178. With `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` set, spawning a teammate no longer needs a setup step, and cleanup happens automatically when the session exits. Before v2.1.178, you asked Claude to create and name a team first, and Claude used the `TeamCreate` and `TeamDelete` tools to set it up and remove it. Both tools no longer exist. The `team_name` input on the Agent tool is accepted but ignored, and the `team_name` field in `TaskCreated`, `TaskCompleted`, and `TeammateIdle` hook payloads carries the session-derived name and is deprecated.

When to use agent teams

Agent teams are most effective for tasks where parallel exploration adds real value. See use case examples for full scenarios. The strongest use cases are:

- **Research and review:** multiple teammates can investigate different aspects of a problem simultaneously, then share and challenge each other's findings
- **New modules or features:** teammates can each own a separate piece without stepping on each

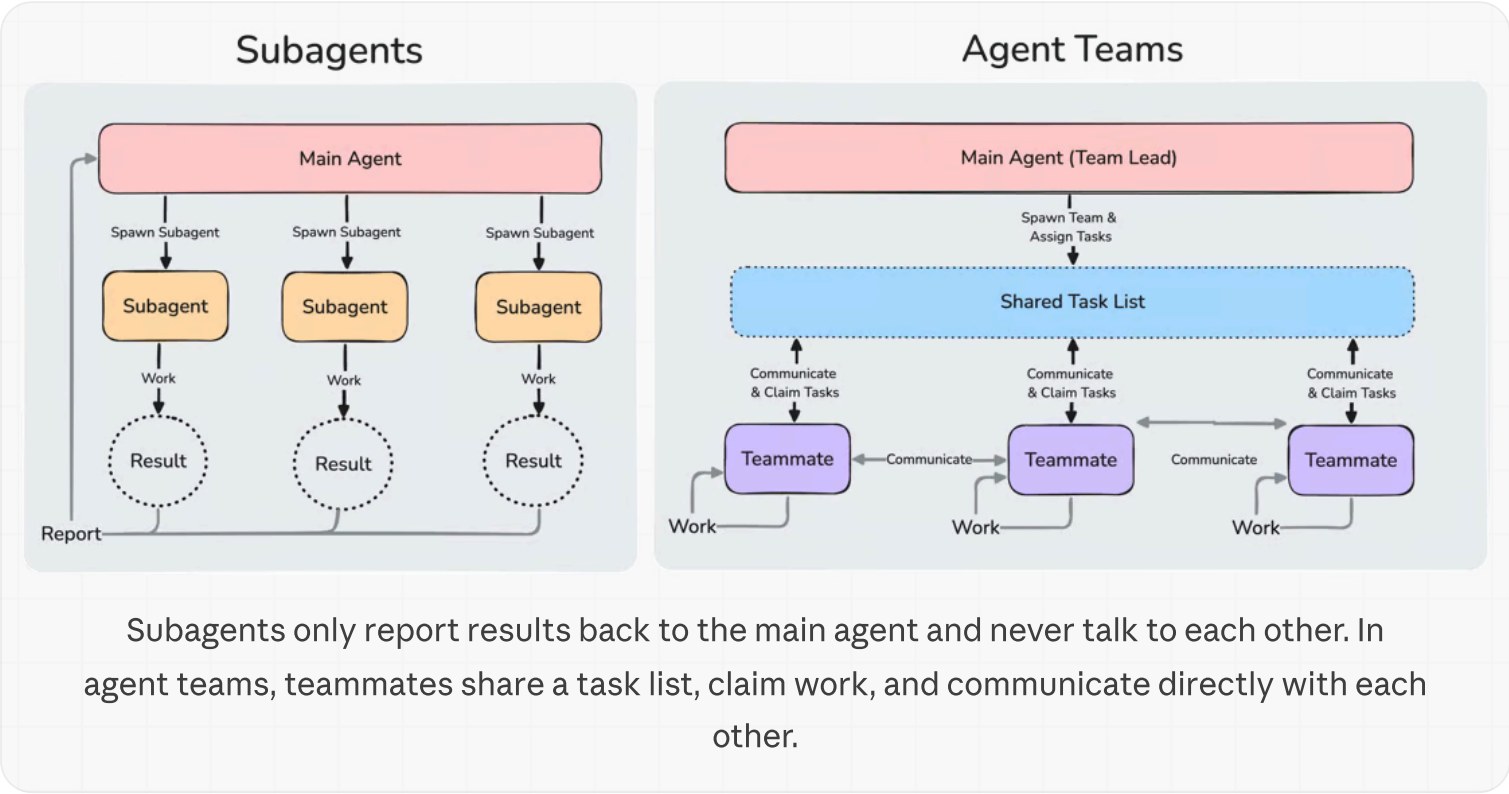
other

- **Debugging with competing hypotheses:** teammates test different theories in parallel and converge on the answer faster
- **Cross-layer coordination:** changes that span frontend, backend, and tests, each owned by a different teammate

Agent teams add coordination overhead and use significantly more tokens than a single session. They work best when teammates can operate independently. For sequential tasks, same-file edits, or work with many dependencies, a single session or subagents are more effective.

Compare with subagents

Both agent teams and subagents let you parallelize work, but they operate differently. Choose based on whether your workers need to communicate with each other:



	Subagents	Agent teams
Context	Own context window; results return to the caller	Own context window; fully independent
Communication	Report results back to the main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion and collaboration

	Subagents	Agent teams
Token cost	Lower: results summarized back to main context	Higher: each teammate is a separate Claude instance

Use subagents when you need quick, focused workers that report back. Use agent teams when teammates need to share findings, challenge each other, and coordinate on their own.

Enable agent teams

Agent teams are disabled by default. Enable them by setting the

`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` environment variable to `1`, either in your shell environment or through [settings.json](#):

settings.json

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Start your first agent team

After enabling agent teams, describe the task and the teammates you want in natural language. Claude spawns them and coordinates work based on your prompt.

This example works well because the three roles are independent and can explore the problem without waiting on each other:

```
I'm designing a CLI tool that helps developers track TODO
comments across
their codebase. Spawn three teammates to explore this from
different angles:
one on UX, one on technical architecture, one playing devil's
advocate.
```

From there, Claude populates a [shared task list](#), spawns teammates for each perspective, has them explore the problem, and synthesizes findings when finished.

The lead's terminal lists teammates in the agent panel below the prompt input. From the panel:

- **Up and down arrows:** select a teammate
- **Enter:** open the selected teammate's transcript and message it directly
- **Escape:** interrupt the selected teammate's current turn

As of v2.1.181, an idle teammate's row hides after 30 seconds and reappears on its next turn. The teammate stays running and addressable while hidden.

If you want each teammate in its own split pane, see [Choose a display mode](#).

Control your agent team

Tell the lead what you want in natural language. It handles team coordination, task assignment, and delegation based on your instructions.

Choose a display mode

Agent teams support two display modes:

- **In-process:** all teammates run inside your main terminal. Use the up and down arrow keys in the agent panel to select a teammate, then press Enter to view it and type to message it directly. Works in any terminal, no extra setup required.
- **Split panes:** each teammate gets its own pane. You can see everyone's output at once and click into a pane to interact directly. Requires tmux, or iTerm2.

❗ `tmux` has known limitations on certain operating systems and traditionally works best on macOS. Using `tmux -CC` in iTerm2 is the suggested entrypoint into `tmux`.

The default is `"in-process"`. Before v2.1.179 the default was `"auto"`, so upgraded sessions that previously opened split panes now stay in one terminal unless you set the mode explicitly. Set `"auto"` to enable split panes when you're already running inside a tmux session or your terminal is iTerm2, falling back to in-process otherwise. The `"tmux"` setting enables split-pane mode and auto-detects whether to use tmux or iTerm2 based on your terminal.

As of v2.1.186, set `"iterm2"` to use iTerm2 native split panes explicitly. This mode requires the [it2 CLI](#) and shows an error with the install command if `it2` is missing. The setup prompt that offers to install `it2` or switch to tmux appears under `"auto"` or `"tmux"` when your terminal is iTerm2 and

tmux is available as a fallback.

To override the default, set `teammateMode` in `~/.claude/settings.json` :

```
{  
  "teammateMode": "auto"  
}
```

To set the mode for a single session, pass it as a flag:

```
claude --teammate-mode auto
```

Split-pane mode requires either **tmux** or iTerm2 with the **it2 CLI**. To install manually:

- **tmux**: install through your system's package manager. See the [tmux wiki](#) for platform-specific instructions.
- **iTerm2**: install the **it2 CLI**, then enable the Python API in **iTerm2 → Settings → General → Magic → Enable Python API**.

Specify teammates and models

Claude decides the number of teammates to spawn based on your task, or you can specify exactly what you want:

```
Spawn 4 teammates to refactor these modules in parallel. Use  
Sonnet for  
each teammate.
```

Teammates don't inherit the lead's `/model` selection by default. To change the model used when the prompt doesn't specify one, set **Default teammate model** in `/config` . Pick **Default (leader's model)** to have teammates follow the lead's current model.

Teammates inherit the lead's **effort level**. In split-pane mode this applies from v2.1.186; earlier versions did not pass the lead's session effort to split-pane teammates.

Require plan approval for teammates

For complex or risky tasks, you can require teammates to plan before implementing. The teammate works in read-only plan mode until the lead approves their approach:

```
Spawn an architect teammate to refactor the authentication
module.
Require plan approval before they make any changes.
```

When a teammate finishes planning, it sends a plan approval request to the lead. The lead reviews the plan and either approves it or rejects it with feedback. If rejected, the teammate stays in plan mode, revises based on the feedback, and resubmits. Once approved, the teammate exits plan mode and begins implementation.

The lead makes approval decisions autonomously. To influence the lead's judgment, give it criteria in your prompt, such as "only approve plans that include test coverage" or "reject plans that modify the database schema."

Talk to teammates directly

Each teammate is a full, independent Claude Code session. You can message any teammate directly to give additional instructions, ask follow-up questions, or redirect their approach.

- **In-process mode:** use the up and down arrow keys in the agent panel to select a teammate, then press Enter to view its session and type to send it a message. Press `x` on a selected teammate to stop it. Press Ctrl+T to toggle the task list.
- **Split-pane mode:** click into a teammate's pane to interact with their session directly. Each teammate has a full view of their own terminal.

Assign and claim tasks

The shared task list coordinates work across the team. The lead creates tasks and teammates work through them. Tasks have three states: pending, in progress, and completed. Tasks can also depend on other tasks: a pending task with unresolved dependencies cannot be claimed until those dependencies are completed.

The lead can assign tasks explicitly, or teammates can self-claim:

- **Lead assigns:** tell the lead which task to give to which teammate

- **Self-claim:** after finishing a task, a teammate picks up the next unassigned, unblocked task on its own

Task claiming uses file locking to prevent race conditions when multiple teammates try to claim the same task simultaneously.

Shut down teammates

To gracefully end a teammate's session, refer to it by name. For example, with a teammate named researcher:

Ask the researcher teammate to shut down

The lead sends a shutdown request. The teammate can approve, exiting gracefully, or reject with an explanation.

The team's shared directories are cleaned up automatically when the session ends, so there's no separate cleanup step. See [Architecture](#) for which directories are removed and which persist for resumed sessions.

Enforce quality gates with hooks

Use **hooks** to enforce rules when teammates finish work or tasks are created or completed:

- **TeammateIdle** : runs when a teammate is about to go idle. Exit with code 2 to send feedback and keep the teammate working.
- **TaskCreated** : runs when a task is being created. Exit with code 2 to prevent creation and send feedback.
- **TaskCompleted** : runs when a task is being marked complete. Exit with code 2 to prevent completion and send feedback.

How agent teams work

This section covers the architecture and mechanics behind agent teams. If you want to start using them, see [Control your agent team](#) above.

How Claude starts agent teams

An agent team forms when the first teammate is spawned, with the main session acting as the lead. There are two ways teammates get spawned:

- **You request teammates:** give Claude a task that benefits from parallel work and explicitly ask for teammates. Claude spawns them based on your instructions.
- **Claude proposes teammates:** if Claude determines your task would benefit from parallel work, it may suggest spawning teammates. You confirm before it proceeds.

In both cases, you stay in control. Claude won't spawn teammates without your approval.

Architecture

An agent team consists of:

Component	Role
Team lead	The main Claude Code session that spawns teammates and coordinates work
Teammates	Separate Claude Code instances that each work on assigned tasks
Task list	Shared list of work items that teammates claim and complete
Mailbox	Messaging system for communication between agents

See **Choose a display mode** for display configuration options. Teammate messages arrive at the lead automatically.

The system manages task dependencies automatically. When a teammate completes a task that other tasks depend on, blocked tasks unblock without manual intervention.

Teams and tasks are stored locally under a session-derived name. The name is `session-` followed by the first eight characters of the session ID:

- **Team config:** `~/.claude/teams/{team-name}/config.json`
- **Task list:** `~/.claude/tasks/{team-name}/`

Claude Code generates both of these automatically at session startup and updates them as teammates join, go idle, or leave. The team config directory is removed when the session ends. The task list directory persists locally and is never uploaded, so resumed sessions keep their tasks. Retention is governed by the same `cleanupPeriodDays` you already control for session transcripts.

The team config holds runtime state such as session IDs and tmux pane IDs, so don't edit it by hand or pre-author it: your changes are overwritten on the next state update.

To define reusable teammate roles, use **subagent definitions** instead.

The team config contains a `members` array with each teammate's name, agent ID, and agent type. Teammates can read this file to discover other team members.

There is no project-level equivalent of the team config. A file like `.claude/teams/teams.json` in your project directory is not recognized as configuration; Claude treats it as an ordinary file.

Use subagent definitions for teammates

When spawning a teammate, you can reference a **subagent** type from any **subagent scope**: project, user, plugin, or CLI-defined. This lets you define a role once, such as a security-reviewer or test-runner, and reuse it both as a delegated subagent and as an agent team teammate.

To use a subagent definition, mention it by name when asking Claude to spawn the teammate:

```
Spawn a teammate using the security-reviewer agent type to audit
the auth module.
```

The teammate honors that definition's `tools` allowlist and `model`, and the definition's body is appended to the teammate's system prompt as additional instructions rather than replacing it. Team coordination tools such as `SendMessage` and the task management tools are always available to a teammate even when `tools` restricts other tools.

❗ The `skills` and `mcpServers` frontmatter fields in a subagent definition are not applied when that definition runs as a teammate. Teammates load skills and MCP servers from your project and user settings, the same as a regular session.

Permissions

Teammates start with the lead's permission settings. If the lead runs with `--dangerously-skip-permissions`, all teammates do too. After spawning, you can change individual teammate modes, but you can't set per-teammate modes at spawn time.

Context and communication

Each teammate has its own context window. When spawned, a teammate loads the same project context as a regular session: CLAUDE.md, MCP servers, and skills. It also receives the spawn prompt from the lead. The lead's conversation history does not carry over.

How teammates share information:

- **Automatic message delivery:** when teammates send messages, they're delivered automatically to recipients. The lead doesn't need to poll for updates.
- **Idle notifications:** when a teammate finishes and stops, they automatically notify the lead.
- **Shared task list:** all agents can see task status and claim available work.
- **Teammate messaging:** send a message to one specific teammate by name. To reach everyone, send one message per recipient.

The lead assigns every teammate a name when it spawns them, and any teammate can message any other by that name. To get predictable names you can reference in later prompts, tell the lead what to call each teammate in your spawn instruction.

Token usage

Agent teams use significantly more tokens than a single session. Each teammate has its own context window, and token usage scales with the number of active teammates. For research, review, and new feature work, the extra tokens are usually worthwhile. For routine tasks, a single session is more cost-effective. See [agent team token costs](#) for usage guidance.

Use case examples

These examples show how agent teams handle tasks where parallel exploration adds value.

Run a parallel code review

A single reviewer tends to gravitate toward one type of issue at a time. Splitting review criteria into independent domains means security, performance, and test coverage all get thorough attention simultaneously. The prompt assigns each teammate a distinct lens so they don't overlap:

```
Spawn three teammates to review PR #142:  
- One focused on security implications  
- One checking performance impact  
- One validating test coverage  
Have them each review and report findings.
```

Each reviewer works from the same PR but applies a different filter. The lead synthesizes findings across all three after they finish.

Investigate with competing hypotheses

When the root cause is unclear, a single agent tends to find one plausible explanation and stop looking. The prompt fights this by making teammates explicitly adversarial: each one's job is not only to investigate its own theory but to challenge the others'.

```
Users report the app exits after one message instead of staying  
connected.  
Spawn 5 agent teammates to investigate different hypotheses. Have  
them talk to  
each other to try to disprove each other's theories, like a  
scientific  
debate. Update the findings doc with whatever consensus emerges.
```

The debate structure is the key mechanism here. Sequential investigation suffers from anchoring: once one theory is explored, subsequent investigation is biased toward it.

With multiple independent investigators actively trying to disprove each other, the theory that survives is much more likely to be the actual root cause.

Best practices

Give teammates enough context

Teammates load project context automatically, including CLAUDE.md, MCP servers, and skills, but they don't inherit the lead's conversation history. See [Context and communication](#) for details. Include task-specific details in the spawn prompt:

```
Spawn a security reviewer teammate with the prompt: "Review the authentication module at src/auth/ for security vulnerabilities. Focus on token handling, session management, and input validation. The app uses JWT tokens stored in httpOnly cookies. Report any issues with severity ratings."
```

Choose an appropriate team size

There's no hard limit on the number of teammates, but practical constraints apply:

- **Token costs scale linearly:** each teammate has its own context window and consumes tokens independently. See [agent team token costs](#) for details.
- **Coordination overhead increases:** more teammates means more communication, task coordination, and potential for conflicts
- **Diminishing returns:** beyond a certain point, additional teammates don't speed up work proportionally

Start with 3-5 teammates for most workflows. This balances parallel work with manageable coordination. The examples in this guide use 3-5 teammates because that range works well across different task types.

Having 5-6 **tasks** per teammate keeps everyone productive without excessive context switching. If you have 15 independent tasks, 3 teammates is a good starting point.

Scale up only when the work genuinely benefits from having teammates work simultaneously. Three focused teammates often outperform five scattered ones.

Size tasks appropriately

- **Too small:** coordination overhead exceeds the benefit
- **Too large:** teammates work too long without check-ins, increasing risk of wasted effort
- **Just right:** self-contained units that produce a clear deliverable, such as a function, a test file, or a review



The lead breaks work into tasks and assigns them to teammates automatically. If it isn't creating enough tasks, ask it to split the work into smaller pieces. Having 5-6 tasks per teammate keeps everyone productive and lets the lead reassign work if someone gets stuck.

Wait for teammates to finish

Sometimes the lead starts implementing tasks itself instead of waiting for teammates. If you notice this:

Wait for your teammates to complete their tasks before proceeding

Start with research and review

If you're new to agent teams, start with tasks that have clear boundaries and don't require writing code: reviewing a PR, researching a library, or investigating a bug. These tasks show the value of parallel exploration without the coordination challenges that come with parallel implementation.

Avoid file conflicts

Two teammates editing the same file leads to overwrites. Break the work so each teammate owns a different set of files.

Monitor and steer

Check in on teammates' progress, redirect approaches that aren't working, and synthesize findings as they come in. Letting a team run unattended for too long increases the risk of wasted effort.

Troubleshooting

Teammates not appearing

If teammates aren't appearing after you ask Claude to spawn them:

- In in-process mode, teammates appear in the agent panel below the prompt input. Use the up and down arrow keys to select one, then press Enter to view it.
- A teammate row that disappeared after sitting idle has been hidden, not stopped. Idle rows hide after 30 seconds and reappear on the teammate's next turn. Send the teammate a message by

name to bring it back.

- Check that the task you gave Claude was complex enough to warrant a team. Claude decides whether to spawn teammates based on the task.
- If you explicitly requested split panes, ensure tmux is installed and available in your PATH:

```
which tmux
```

- For iTerm2, verify the `it2` CLI is installed and the Python API is enabled in iTerm2 preferences.

Too many permission prompts

Teammate permission requests bubble up to the lead, which can create friction. Pre-approve common operations in your **permission settings** before spawning teammates to reduce interruptions.

Teammates stopping on errors

Teammates may stop after encountering errors instead of recovering. Check their output by selecting the teammate in the agent panel and pressing Enter in in-process mode, or by clicking the pane in split mode, then either:

- Give them additional instructions directly
- Spawn a replacement teammate to continue the work

Lead shuts down before work is done

The lead may decide the team is finished before all tasks are actually complete. If this happens, tell it to keep going. You can also tell the lead to wait for teammates to finish before proceeding if it starts doing work instead of delegating.

Orphaned tmux sessions

If a tmux session persists after the Claude Code session ends, it may not have been fully cleaned up. List sessions and end the one created by the team:

```
tmux ls
tmux kill-session -t <session-name>
```

Limitations

Agent teams are experimental. Current limitations to be aware of:

- **No session resumption with in-process teammates:** `/resume` and `/rewind` do not restore in-process teammates. After resuming a session, the lead may attempt to message teammates that no longer exist. If this happens, tell the lead to spawn new teammates.
- **Task status can lag:** teammates sometimes fail to mark tasks as completed, which blocks dependent tasks. If a task appears stuck, check whether the work is actually done and update the task status manually or tell the lead to nudge the teammate.
- **Shutdown can be slow:** teammates finish their current request or tool call before shutting down, which can take time.
- **One team per session:** a session has exactly one team, scoped to that session. You can't create additional named teams or share a team across sessions.
- **No nested teams:** teammates cannot spawn their own teammates. Only the lead can manage the team.
- **Lead is fixed:** the main session is the lead for its lifetime. You can't promote a teammate to lead or transfer leadership.
- **Permissions set at spawn:** all teammates start with the lead's permission mode. You can change individual teammate modes after spawning, but you can't set per-teammate modes at spawn time.
- **Split panes require tmux or iTerm2:** the default in-process mode works in any terminal. Split-pane mode isn't supported in VS Code's integrated terminal, Windows Terminal, or Ghostty.



`CLAUDE.md` **works normally:** teammates read `CLAUDE.md` files from their working directory. Use this to provide project-specific guidance to all teammates.

Next steps

Explore related approaches for parallel work and delegation:

- **Lightweight delegation:** subagents spawn helper agents for research or verification within your

session, better for tasks that don't need inter-agent coordination

- **Manual parallel sessions:** Git worktrees let you run multiple Claude Code sessions yourself without automated team coordination
- **Compare approaches:** see the subagent vs agent team comparison for a side-by-side breakdown